

Appendix: Measurement Methodology

Reproduction, evidence scope, and the limits of the measurements

Matan Cohen · Yuval Kogan | Technion | HWSW Final Project

A1. Timing and provenance

The headline results use the course QEMU/KVM VM, Ubuntu 22.04, release CPython 3.10.12 and pyperformance 1.14.0 with pypert 2.10.0. Each JSON contains 120 measured values (40 value-bearing worker runs, three values each); calibration is separate. Reported uncertainty is sample standard deviation across those values, not a confidence interval. Use the full-precision JSON means to calculate ratios, rather than rounded display values.

Comparison	Files in the results directory
Nbody stock / Python	baseline_nbody.json / optimized_nbody.json
Nbody Python / native	vm_rerun_20260907/fresh2_nbody_{python,native}.json
Pyflate all three tiers	vm_release_20260907/pyflate_{stock,python,native}.json

Nbody's stock/optimized and Python/native pairs are separate experiments. Its older stock/optimized JSONs lack backend metadata; the latest pair explicitly records python and native. Pyflate's three tiers were rerun sequentially on one fixed guest CPU, using the same release interpreter and the refactored Rust source built on that VM. These three files supply its updated headline results. Earlier captures remain available for comparison.

Development evidence is labeled separately. Pyflate's ablation and cProfile tables come from Windows / CPython 3.12.6. Nbody's detailed unrolling sweep and pyflate's older 25× native kernel example come from WSL2 / CPython 3.10.21. Different machines, interpreters and estimators prevent using these absolute timings as VM results. Likewise, a paired median speedup need not equal the ratio of two independently reported minimum times. Bytecode counts collected with `sys.monitoring` also depend on the CPython version and counting workload; they are not portable instruction counts for the VM's interpreter.

Reproducing the comparisons

From the repository root, compare saved data without changing it:

```
python3 -m pypert compare_to results/baseline_nbody.json \
results/optimized_nbody.json
python3 -m pypert compare_to results/vm_release_20260907/pyflate_python.json \
results/vm_release_20260907/pyflate_native.json
```

For fresh native/Python comparisons, build and install the extension first, then run both back ends against it:

```
./tools/build_wheel.sh nbody --record /tmp/nbody-wheel.json
HWSW_BACKEND=python python3 benchmarks/bm_nbody/run_benchmark.py \
--rigorous --inherit-environ HWSW_BACKEND -o /tmp/nbody-python.json
HWSW_BACKEND=native python3 benchmarks/bm_nbody/run_benchmark.py \
--rigorous --inherit-environ HWSW_BACKEND -o /tmp/nbody-native.json
```

Build and install must name the same file. `maturin build --release` writes to `target/wheels/`, while `rust/<crate>/wheels/` holds the prebuilt wheel committed for the course VM; both are version 0.1.0, so `pip install wheels/*.whl` after a fresh build silently installs the saved binary instead of what was just compiled, and every number downstream then describes the saved binary. `tools/build_wheel.sh` builds into an empty directory, requires exactly one wheel to appear there, installs that path, and records what Python subsequently imported – module path, module SHA-256, wheel SHA-256, crate source hashes and toolchain versions.

Use new output names for each experiment. Repeat with `bm_pyflate` for decompression. `pyperformance run` creates its own environment; direct benchmark execution avoids assuming that environment contains the extension wheels. Native mode fails when a wheel is unavailable. The runner scripts at the repository root provide the full stock/optimized workflow; they write to a fresh `results/runs/<UTC stamp>_<bench>/` directory so a rerun cannot overwrite the preserved captures this appendix cites.

Back-end selection is enforced in three places, not requested in one. Each `run_benchmark.py` appends `HWSW_BACKEND` to `pypert`'s inherited environment after parsing its arguments, so worker processes receive it even when the caller omits `--inherit-environ` – which `pyperformance run` gives no way to supply. Every

measured and profiled invocation in the runners is pinned, including `perf record`, `py-spy` and `perf stat` on both the stock and optimized sides. Finally the native stage asserts, on each JSON it produced, both the back end that ran (`hwsw_backend`) and the request the workers actually saw (`hwsw_backend_requested`), so a run whose environment never reached the workers fails even if the fallback happened to choose the right path. Result metadata also records the loaded extension's path and SHA-256, because a crate version alone does not distinguish two builds.

Correctness

Nbody comparisons check all state components and energy after the same initial conditions and 20,000 steps. `Exact ==` is the observed result for the shipped Python and native kernels; a tolerance pass alone does not establish equality, and numerical equality is not a general byte-representation test (for example, signed zeros). Pyflate checks the decoded bytes against `bz2.decompress`, the original MD5, the intermediate L-vector and the ending bit position.

Exact and tolerance results are separate verdicts, and the declared contract decides the exit status.

The two claims are different, so collapsing them would let a rounding-changing edit pass a checker while the reports still claimed equality.

Checker	Contract	Failure behaviour
<code>dev/nbody/verify.py tiers</code>	Tolerance	T3's <code>sqrt</code> variants reorder arithmetic deliberately; scored against the stated thresholds.
<code>dev/nbody/verify.py landed benchmark</code>	Exact	Nonzero exit unless bit-identical to stock; the tolerance verdict is still printed.
<code>dev/nbody/rs_check.py</code>	Exact (default)	Nonzero exit unless bit-identical. - - tolerance moves the gate and says so in the output.
<code>dev/pyflate/rs_check.py</code>	Byte-exact	Nonzero exit unless the L-vector, ending bit position, output bytes and MD5 all match.

A2. Matched Python/native CPU counters

The counter tables use `report/measure_native_counters.py` and the raw files in `results/vm_release_20260907/counters/`. Release CPython 3.10.12 executes the same benchmark source in explicit Python and native modes, on one fixed guest CPU. Each configuration has three runs per event pass; backend order alternates between rounds. Every run warms up before measurement. Nbody resets state afterwards and executes 64 iterations of energy, 20,000 steps and energy; pyflate executes 16 complete decompressions. Output checks follow the timer.

Counter scope. `perf stat --delay=-1 --control fifo:...` starts with counters disabled. The worker enables counting after imports, setup, warmup and garbage collection, waits for acknowledgement, then times the loop. It disables counting afterwards and waits for a second acknowledgement. Counts include the small control-boundary overhead but exclude setup and output validation. The internal elapsed timer covers the loop only. Both core and generic cache events request user mode (:u); separate event passes are separate executions.

Pass	Events and validation
Core	Cycles, instructions, branches, branch misses: retained; reported running time 100%.
Cache	Generic cache references/misses: retained; reported running time 100%.
L1 events	Not requested: an earlier capture returned invalid zero load counts. No L1 miss-rate claim is made.

The summarizer checks source SHA-256 hashes, requested backends, output SHA-256 equality across all 24 runs, nonzero denominators and counter running time. A separate dispatch check (`report/check_pyflate_backend.py`) observed zero native calls for Python and one per block for native/auto, with identical MD5 and output length. The measured pyflate extension is the refactored build; its binary hash, Rust toolchain version and source hashes are recorded in `vm_release_20260907/protocol.json`. Correctness logs identify the loaded extension path.

Reported counts are normalized per completed benchmark iteration. Each displayed value is the median of three observations, including rates calculated **within each run**. Thus a ratio of displayed median counts may differ slightly from the displayed median rate. `summary.json` also retains the individual values and minimum/maximum. Three runs support a descriptive comparison, not a formal significance claim; the rigorous 120-value timing experiments remain the headline evidence.

Interpretation. IPC is retired instructions per counted cycle, not work completed per cycle. Removing interpreter instructions can lower IPC while improving runtime substantially. Generic cache events do not represent all memory accesses or identify stall cycles, a specific cache level, or DRAM latency. Nbody's very small miss counts are particularly sensitive to boundary overhead and run-to-run variation. Pyflate's larger remaining counts cannot be assigned to BWT without an isolated phase measurement.

To regenerate the validated summaries from the preserved capture:

```
python3 report/summarize_counters.py \
  --directory results/vm_release_20260907/counters
```

For a new capture on Linux, choose a fresh output directory:

```
python3 report/measure_native_counters.py --repo /path/to/repo \
  --out /tmp/new-native-counters
```

A3. Earlier phase counters: evidence and limits

`dev/pyflate/phase_cpi.py` repeats one phase after preparing its input. The internal wall timer covers the repeated phase; `perf stat` wraps the **entire process**, including imports and input preparation. The values below reproduce `results/pyflate_phase_cpi.txt`; wall times are divided by repeat count and rounded. These are phase timing and aggregate-counter measurements, not a CPI stack decomposing execution into stall causes.

Phase	Repeats	ms/iter	IPC	Cache misses / refs
Native decode	300	3.304	1.49	1.284%
Whole inverse BWT	20	137.720	2.92	2.601%
Chase loop	30	47.820	2.41	1.831%
RLE4	100	23.836	2.89	1.623%

IPC uses instructions:u / cycles:u; the generic cache events were collected without that user-only modifier. Ratios are process-wide. Cache events do not identify which access missed, the latency it incurred, or how much useful work overlapped it. Therefore we do not infer an exact DRAM-access interval, or assign all cycles per traversal step to interpreter overhead.

The chase loop performs `end = T[end]`, reads `L[end]`, writes an output byte and runs Python loop machinery. Its setup builds `T` outside the wall timer but inside the counter interval. The `L`-vector contains 336,184 bytes; `T` is a Python list of indices, whose list slots and integer objects add memory. The final 399,360-byte output size is not the traversal table size. Whole BWT includes substantial table-construction work as well as the dependent traversal.

PMU and profiling configuration

- **Counting:** the VM experiments observed usable cycle/instruction counts and zeros when too many hardware events were requested together. Scripts split events into passes of at most four. Check actual counts and enabled/running time rather than assuming multiplexing.
- **Sampling:** tested `cycles -F` invocations produced zero samples, whereas fixed-period `cycles -c` and software `cpu-clock` sampling worked. The experiment demonstrates a working configuration; it does not establish the kernel's internal cause or a universal KVM limitation. Core and reference cycles are distinct events even when counts are close.
- **Call chains:** `--call-graph dwarf,16384` produced usable debug-build stacks where frame-pointer unwinding failed. Debug-build profiles describe that build's costs and are not used as release timings. Python-frame sampling uses `py-spy` separately.
- **Sample rate:** check `perf_event_max_sample_rate` after system tuning. A rate of 1 Hz was observed in the tuned setup and suppresses useful profiles. Do not assume it persists after a reboot or is the default on another machine.

Example counter command (whole-process scope):

```
HWSW_BACKEND=native perf stat \
  -e cycles:u,instructions:u,cache-references,cache-misses -- \
  python3 dev/pyflate/phase_cpi.py --phase chase --repeat 30
```

To establish a native BWT memory bottleneck, measure a native implementation, isolate or subtract setup with a justified method, and sweep working-set size. The current counters are insufficient for a measured SRAM-versus-DRAM accelerator comparison.

A4. Full profiles and release verification

The reports show full original flame graphs beside enlarged context crops. `report/make_figures.py` copies every recorded frame rectangle at its original coordinates; it does not merge functions, filter startup, cap stack depth or renormalize widths. Numbered outlines mark the same frame in the overview and its detail crop. Selected-frame titles, coordinates, sample weights and source filenames are recorded in `report/fig/profile_counts.json`.

Stock nbody uses the C-frame debug profile to expose interpreter operations. Optimized nbody and both pyflate figures use Python-frame py-spy profiles. The tall C graph needs a small overview to retain its complete context; the enlarged regions carry readable local detail. Original SVGs in `results/` remain available for arbitrary zoom and inspection.

Widths show **inclusive** samples for a call path, relative to the original whole profile. The C graph's weights sum sampled event periods, not a count of individual interrupts. A function may appear at several locations; the highlighted occurrence is not its aggregate self time. Children overlap their parents, so widths on different rows must not be added. Independent graphs are independently normalized: a narrower-looking stack is not evidence of an absolute speedup. Python-frame views have only a few hundred samples and are used to locate work, not resolve small percentage differences.

Older trimmed views in `results/` remove startup or cap stack depth. The reports use the `*_full.svg` originals, including the four C-level flame_{nbody,pyflate}_{stock,opt}_full.svg profiles. Flat self-time evidence is in `perf_report_*.txt`; headline timings come from JSON.

VM correctness checks, 2026-09-07

Nbody's shipped Python kernel and its Rust kernel matched stock state and energy exactly after 20,000 steps. Pyflate's Python and hybrid outputs matched bz2; the L-vector and ending bit position matched too. Benchmark and checker source hashes matched the local copies.

Fresh VM rerun

The four earlier rerun JSONs are preserved in `results/vm_rerun_20260907/`. Each contains **120 measured values from 40 workers**, with the expected backend metadata. A rigorous job already contains multiple workers and values; these runs have the same sample count as the older headline files. Benchmark source hashes match this checkout. These pyflate measurements used the previously installed extension, before the refactored source was rebuilt.

Earlier VM rerun	Python mean $\pm$ SD	Native mean $\pm$ SD	Ratio
Nbody	142.19 $\pm$ 4.48 ms	9.477 $\pm$ 0.030 ms	15.00×
Pyflate	285.64 $\pm$ 3.04 ms	171.89 $\pm$ 2.20 ms	1.66×

The nbody native comparison now uses this verified rerun. Pyflate uses the subsequent three-tier experiment in `vm_release_20260907/`, which builds the current crate with `cargo build --locked --release` for CPython 3.10.12. Ten Rust tests and seven blocks across five fixtures pass; both decode APIs, traces and exact ending offsets are checked. The isolated build is selected through inherited `PYTHONPATH`; installed wheels and the existing VM checkout remain intact. `report/summarize_refresh.py` validates source hashes, backend metadata and sample counts. Independent captures are retained separately.

A5. Timing distributions and worker clustering

mean $\pm$ SD describes a symmetric spread of independent observations. Neither assumption holds exactly here, and the table below says by how much. It is produced from the same result JSONs, without re-timing anything:

```
python3 report/summarize_distribution.py results/baseline_nbody.json \
  results/optimized_nbody.json \
  results/vm_rerun_20260907/fresh2_nbody_{python,native}.json \
  results/vm_release_20260907/pyflate_{stock,python,native}.json
```

Shape. Every distribution is right-skewed: the median sits below the mean and the maximum is far from both, which is the usual signature of occasional interference rather than a symmetric measurement error. The interquartile range is therefore the more informative spread, and it is between 0.2% and 1.4% of the median in every configuration.

Configuration	Median	IQR	p95	Max	Mean
Nbody stock	230.29	3.54	236.75	249.82	231.23
Nbody optimized	140.02	0.77	147.22	164.50	141.28
Nbody Python (rerun)	140.48	2.60	150.48	166.79	142.19
Nbody native	9.466	0.017	9.558	9.578	9.477
Pyflate stock	1128.01	12.97	1157.10	1173.01	1129.89
Pyflate Python	287.84	3.62	293.49	304.39	288.00
Pyflate native	172.91	2.35	177.96	181.16	173.59

Grouping. The 120 values are not 120 independent observations. They come from 40 worker processes, three values each, and values from one worker share that process's memory layout, CPU placement and page-cache state. A one-way variance decomposition splits the total into a between-worker and a within-worker component; the between share is the intraclass correlation (ICC), and the variance of the mean is inflated by the design effect $\text{deff} = 1 + (m - 1) * \text{ICC}$, with $m = 3$ values per worker. (Set as code rather than Typst math, for the reason given in A6.) The block below is the tool's own output, verbatim:

clustering	SD-betw	SD-with	ICC	deff	SE naive	SE clust
baseline_nbody	3.269	1.034	0.909	2.82	0.3106	0.5214
optimized_nbody	4.253	0.688	0.975	2.95	0.3900	0.6698
fresh2_nbody_python	3.915	2.230	0.755	2.51	0.4087	0.6475
fresh2_nbody_native	0.018	0.024	0.364	1.73	0.0028	0.0037
pyflate_stock	9.684	8.280	0.578	2.16	1.1575	1.6993
pyflate_python	0.000	3.276	0.000	1.00	0.2936	0.2936
pyflate_native	1.006	1.902	0.219	1.44	0.1961	0.2351

All figures in ms. SE naive treats the 120 values as independent; SE clust is the honest standard error.

Effective sample size is $120 / \text{deff}$: 42.6 and 40.7 for nbody's stock and optimized runs, 47.8 and 69.5 for the nbody rerun pair, and 55.6, 120.0 and 83.3 for pyflate's three tiers.

Nbody's two configurations are almost entirely between-worker (ICC 0.91 and 0.98): a worker's three values agree closely with each other and less well with another worker's, so the effective sample size is about 41–43 rather than 120 and the standard error of the mean is roughly 1.7x what independence would give. Pyflate is milder, and its optimized Python tier shows no detectable worker effect at all.

What this does and does not change. Every reported speedup is one to two orders of magnitude larger than the corrected standard errors, so no headline conclusion moves. What it does change is the reading of small differences: a gap of a few tenths of a millisecond between two nbody configurations is not resolvable at this sample size, whatever the naive SD suggests. A negative between-worker variance estimate is reported as zero, which is the statement that no worker effect is detectable, not that the workers are provably identical. Three values per worker is a small basis for an ICC estimate; these figures describe these runs and are not offered as properties of the guest.

A6. Flat function tables

Flame graphs show where time goes but are a poor place to read a number off: every width is inclusive of children, one function may appear on several call paths, and print crops can clip labels. `report/summarize_profiles.py` produces the companion table, with both percentages named, from the same recorded files the figures use:

```
python3 report/summarize_profiles.py --top 10
python3 report/summarize_profiles.py --only perf_report_nbody_stock.txt \
  --group 'list access=^(list_|listiter_|PyNumber_AsSsize_t|PyLong_AsSsize_t)'
```

The two profile kinds are read differently because they record different things. For `perf report --stdio` the **Children** and **Self** columns are read directly. For a py-spy flame graph, inclusive time is a frame's own sample count and self time is that count minus the counts of the frames stacked directly on it; frames are aggregated per function and file, since py-spy labels each frame with whichever line was executing.

Prose that adds up a family of symbols must quote a number this prints, together with the pattern it came from. An earlier draft of the nbody report gave 14.4% for list access; the recorded profile yields **11.95%** over `^(list_|listiter_|PyNumber_AsSsize_t|PyLong_AsSsize_t)`, and float object handling is the larger group at 16.30% over `^(float_|PyFloat_)`. The tool lists every contributing symbol so the sum can be checked. Summing self time across perf rows is correct even though perf emits some symbols twice – an (inlined) entry carries inclusive time and zero self, and separate entries for one symbol are separate contributions; the self column over a whole report sums to within half a percent of 100%, which is the check. Saved outputs: `results/timing_distributions.txt`, `results/profile_functions.txt`, and the machine-readable `report/fig/timing_distributions.json` and `report/fig/profile_functions.json`.

A note on the text companions

The .txt files beside each PDF are produced by `pdftotext -layout`, which reconstructs rows from glyph positions and is not reliable on wide numeric tables. Two failures were found and fixed: an ablation table whose narrow last cell caused every “added cost” to be printed one row too high, and a profile table with a spanning header row whose values were likewise shifted. In both the PDF was correct and only the text companion was wrong, silently. Typst math is a second hazard: its italic letters export as byte sequences that are not valid UTF-8, and a display fraction loses its denominator, so formulas are set as code instead. `report/check_txt_tables.py` now runs in the build. For every table row in every `report_*.typ` it requires the row's label and its own cell values to appear in that order, close together, in the exported text, and it rejects a companion that is not valid UTF-8.

References: Python [profile semantics](#) [version-specific bytecode repository](#) and [raw data](#).